

Mentor Instructions

1. Join 5 minutes early, check screen share, audio, and that both `seed_data.py` and the solution `main.py` file are open and ready locally in separate tabs/windows — use the solution as a silent reference throughout, not to share with learners, but to know what correct output looks like at every step.
2. Before the session, run the complete solution file end-to-end yourself at least once to anticipate where AI output may deviate from expected results. Also keep the list of 5 prompts in a separate doc for reference.
3. Keep a backup copy of `seed_data.py` untouched — if inventory values get accidentally modified during testing, you have a clean reset.
4. Start with a 5-minute welcome — set expectations, confirm learners have watched Week 1 videos, and briefly outline the session flow.
5. Begin by explaining why AI is being used as a coding assistant - to quickly convert interconnected data models into runnable logic. It helps accelerate development, easily adapt to changing business rules. Before coding, walk learners through `seed_data.py` and its five tables — Recipes, Inventory, Orders, Restock, and Status — so they understand the data foundation of the application.
6. Walk through the Business Context — explain cloud kitchen margins (20–30% aggregator cut, 5–15% contribution margin) and why operational inefficiencies matter financially. Cover stockouts, food waste, and ranking impact.
7. Present the Problem Statement — explain the 6 core problems: silent stockouts, manual menu updates, no expiry tracking, inefficient restocking, no partial fulfillment logic, and no structured procurement planning.
8. Open a blank `main.py`. Use Prompt 1 (Step 1) — before doing so, explain to learners what this prompt is trying to achieve: establishing data connectivity so the application can see all tables it needs to work with. Run it and verify all tables print correctly before moving on.
9. Use Prompt 2 (Step 2) — before prompting, explain to learners that this step defines the kitchen's rhythm: pulling an order, matching it to a recipe, and

calculating ingredient needs. Run and verify that each order maps to its recipe and ingredient requirements are calculated correctly.

10. Use Prompt 3 (Step 3) — explain to learners that this step adds the fulfillment decision: if stock is available the order gets delivered and inventory is deducted; if not, the order is flagged and missing items go to restock. Run and cross-check Delivered/Not Delivered status against the expected output in the solution file.
11. Use Prompt 4 (Step 4) — explain to learners why cumulative deduction matters: each order must check against what's left after the previous order, not the original stock, to reflect real kitchen conditions. Run and verify that Order 2 onwards reflects remaining stock from prior orders, not the original inventory.
12. Use Prompt 5 (Step 5) — explain to learners that this step automates the restock decision using three business rules tied to expiry and stock levels, removing the need for manual intervention. Run and verify the three restock conditions (Expiring soon / Out of stock / Running low) trigger correctly.
13. After each prompt, compare output against the solution file before moving to the next step. If the AI generates logically correct but differently structured code, don't force a rewrite — validate the output against expected results and move on. If the output doesn't match, try adding one specific constraint to the same prompt rather than rewriting it entirely — for example, "ensure ingredient names match exactly as in seed_data.py."
14. Watch out for common failure points: ingredient name mismatches between Orders and Recipes (case sensitivity), date format errors in expiry logic, and cumulative deduction not carrying forward correctly to the next order.
15. After Step 3 (Prompt 3), do a quick manual check on Order 5 (45 Chicken Burgers for Taco Bell) — this is the most likely order to expose inventory shortfall bugs and cumulative deduction errors.
16. If learners fall behind, pause at the end of each Part — this keeps the session paced without losing the group.
17. Close with a 10-minute Q&A, then a 5-minute summary covering the key takeaway: this is structured problem-solving with AI aligned to business outcomes, not just writing code with AI.